

UNIT 2 & UNIT 4 – EXAM PREPARATION ANSWERS

UNIT 2 – JAVA CONCEPTUAL FOUNDATIONS AND CAPABILITIES

PART B – Short Answers

1. What is a Package? How do you import a package into a Java program? Give the syntax.

A package is a collection of related classes, interfaces and sub-packages. It is used to organize Java programs, avoid name conflicts and provide access protection.

Syntax to create a package:

```
package packagename;
```

Syntax to import:

```
import packagename.ClassName;
```

or

```
import packagename.*;
```

Example:

```
import java.util.Scanner;
```

```
Scanner sc = new Scanner(System.in);
```

2. List the differences between an abstract class and an interface.

Abstract class:

- Declared using abstract keyword.
- Can contain abstract and concrete methods.
- Can have constructors and instance variables.
- A class extends only one abstract class.

Interface:

- Declared using interface keyword.
- Mainly defines a contract for classes.

- A class can implement multiple interfaces.
- Methods are public by default in the traditional interface model.

Thus, interfaces are useful for achieving multiple inheritance of type.

3. Differentiate between checked and unchecked exceptions with examples.

Checked exceptions are checked by the compiler at compile time. They must be handled using try-catch or declared using throws. Example: IOException, SQLException.

Unchecked exceptions occur at runtime and are subclasses of RuntimeException. Example: ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException.

Example:

```
int x = 10 / 0; // ArithmeticException
```

4. What is a user-defined exception? How is it created in Java?

A user-defined exception is an exception created by the programmer for a specific application condition.

Steps:

1. Create a class extending Exception.
2. Provide a constructor.
3. Create an object of the exception.
4. Use throw to raise it.

Example:

```
class AgeException extends Exception {  
    AgeException(String msg) {  
        super(msg);  
    }  
}  
  
throw new AgeException("Invalid age");
```

5. Define multithreading. List any two advantages.

Multithreading is the process of executing multiple threads concurrently within a program.

Advantages:

1. Better CPU utilization.
2. Faster execution of independent tasks.
3. Improved responsiveness.
4. Efficient resource sharing.

6. List the different states in the life cycle of a Java thread.

The main thread states are:

1. NEW – thread is created but not started.
2. RUNNABLE – thread is ready/running.
3. BLOCKED – waiting to acquire a lock.
4. WAITING – waiting indefinitely for another thread.
5. TIMED_WAITING – waiting for a specified time.
6. TERMINATED – execution is completed.

7. What is autoboxing and unboxing? Give an example.

Autoboxing is the automatic conversion of a primitive value into its corresponding wrapper object. Unboxing is the automatic conversion of a wrapper object into a primitive value.

Example:

```
int n = 10;
```

```
Integer obj = n;    // autoboxing
```

```
int x = obj;        // unboxing
```

8. What are generics? State one advantage.

Generics allow classes, interfaces and methods to operate on specified types.

Example:

```
ArrayList<Integer> list = new ArrayList<>();
```

Advantage: Generics provide compile-time type safety and reduce the need for explicit type casting.

9. What are annotations in Java? Give any two built-in annotations.

Annotations provide metadata about program elements such as classes and methods. They do not directly change the program's basic logic.

Examples:

@Override – indicates that a method overrides a superclass method.

@Deprecated – indicates that an element should no longer be used.

10. Differentiate throw and throws in Java.

throw:

- Used to explicitly throw an exception.
- Used inside a method or block.
- Throws one exception object at a time.

Example:

```
throw new ArithmeticException("Error");
```

throws:

- Used in a method declaration.
- Declares exceptions that may be thrown by the method.
- Can declare multiple exceptions.

Example:

```
void read() throws IOException { }
```

PART C – Long Answers / Programming

1. Abstract Classes and Interfaces – Multiple Interface Implementation

An abstract class is a class declared with abstract keyword. It may contain abstract and concrete methods. An interface specifies a contract that a class must implement. Java allows a class to implement more than one interface.

Program:

```
interface Printable {  
    void print();  
}  
  
interface Showable {  
    void show();  
}  
  
class Demo implements Printable, Showable {  
    public void print() {  
        System.out.println("Print method");  
    }  
    public void show() {  
        System.out.println("Show method");  
    }  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d.print();  
        d.show();  
    }  
}
```

Output:

Print method

Show method

Conclusion: Multiple interfaces allow a class to inherit multiple contracts.

2. Exception Handling in Java

Exception handling prevents abnormal termination and allows the program to handle runtime errors.

Keywords:

- try – contains risky code.
- catch – handles an exception.
- finally – executes whether exception occurs or not.
- throw – explicitly throws an exception.
- throws – declares possible exceptions in a method.

Program:

```
class ExceptionDemo {  
    static void check(int a) throws Exception {  
        if (a == 0)  
            throw new Exception("Zero is not allowed");  
    }  
    public static void main(String[] args) {  
        try {  
            check(0);  
        } catch (Exception e) {  
            System.out.println(e.getMessage());  
        } finally {  
            System.out.println("Finally block executed");  
        }  
    }  
}
```

Output:

Zero is not allowed

Finally block executed

3. Java Thread Model and Two Ways of Creating Threads

A thread is a lightweight unit of execution. Java supports concurrent execution using threads.

Method 1 – Extending Thread:

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

Method 2 – Implementing Runnable:

```
class MyTask implements Runnable {  
    public void run() {  
        System.out.println("Runnable thread is running");  
    }  
    public static void main(String[] args) {  
        Thread t = new Thread(new MyTask());  
        t.start();  
    }  
}
```

start() creates/schedules a new thread and the run() method contains the task executed by that thread.

4. Autoboxing, Unboxing and Wrapper Classes

Wrapper classes provide object representations of primitive data types.

byte – Byte

short – Short

int – Integer

long – Long

float – Float

double – Double

char – Character

boolean – Boolean

Autoboxing:

```
int a = 10;
```

```
Integer x = a;
```

Unboxing:

```
Integer y = 20;
```

```
int b = y;
```

Program:

```
class WrapperDemo {  
    public static void main(String[] args) {  
        int n = 25;  
        Integer obj = n;    // autoboxing  
        int value = obj;    // unboxing  
        System.out.println(obj);  
        System.out.println(value);  
    }  
}
```

```
}  
}
```

Output:

25

25

5. Generic Programming in Java

Generics allow a class or method to work with different data types while maintaining type safety.

Generic class and generic method:

```
class Box<T> {  
    T value;  
    Box(T value) {  
        this.value = value;  
    }  
    T getValue() {  
        return value;  
    }  
    static <E> void display(E data) {  
        System.out.println(data);  
    }  
    public static void main(String[] args) {  
        Box<Integer> b1 = new Box<>(100);  
        Box<String> b2 = new Box<>("Java");  
        System.out.println(b1.getValue());  
        System.out.println(b2.getValue());  
    }  
}
```

```
        display(25);  
        display("Generics");  
    }  
}
```

Advantages:

1. Compile-time type safety.
2. Reduces explicit casting.
3. Promotes code reusability.
4. Errors are detected earlier.

UNIT 4 – I/O AND GUI PROGRAMMING IN JAVA

PART B – Short Answers

1. List the classes used in Java to read data from the console. Give Scanner syntax.

Common console input classes are Scanner, BufferedReader with InputStreamReader, and Console.

Scanner syntax:

```
import java.util.Scanner;
```

```
Scanner sc = new Scanner(System.in);
```

Example:

```
int n = sc.nextInt();
```

```
String s = sc.nextLine();
```

2. Differentiate between byte streams and character streams.

Byte streams handle raw binary data and use InputStream/OutputStream classes.

Examples: FileInputStream and FileOutputStream.

Character streams handle character/text data and use Reader/Writer classes. Examples: FileReader and FileWriter.

Byte streams are suitable for binary files; character streams are suitable for text files.

3. What is **BufferedReader**? Give syntax with example.

BufferedReader reads text efficiently by buffering characters.

Syntax:

```
BufferedReader br = new BufferedReader(new FileReader("input.txt"));
```

Example:

```
String line = br.readLine();
```

```
System.out.println(line);
```

```
br.close();
```

4. List any three **Scanner** methods with purpose.

1. **nextInt()** – reads an integer.
2. **nextDouble()** – reads a double.
3. **nextLine()** – reads an entire line of text.

Other methods include **next()**, **nextFloat()**, **nextLong()**, etc.

5. What is the use of **PrintWriter**? Give an example.

PrintWriter is used to print formatted character data to a stream.

Example:

```
import java.io.*;
```

```
class PrintDemo {
```

```
    public static void main(String[] args) {
```

```
        PrintWriter pw = new PrintWriter(System.out);
```

```
        pw.println("Hello Java");
```

```
        pw.close();
```

```
    }
```

```
}
```

6. Steps involved in writing data to a text file. List classes.

Steps:

1. Create/open a file.
2. Create a character output stream such as `FileWriter`.
3. Optionally wrap it with `BufferedWriter`.
4. Write data.
5. Close the stream.

Classes: `FileWriter`, `BufferedWriter`, `PrintWriter` and `File`.

7. What is the use of File class? List three methods.

The `File` class represents a file or directory path and provides methods to obtain information about it.

Methods:

1. `exists()` – checks whether the file exists.
2. `length()` – returns file size in bytes.
3. `lastModified()` – returns the last modified time.

Other methods include `getName()`, `delete()`, `isFile()`, `isDirectory()`.

8. Differentiate FileReader/FileWriter and FileInputStream/FileOutputStream.

`FileReader/FileWriter` are character streams used mainly for text data.

`FileInputStream/FileOutputStream` are byte streams used for binary/raw data.

`FileReader` → reads characters.

`FileWriter` → writes characters.

`FileInputStream` → reads bytes.

`FileOutputStream` → writes bytes.

9. What is the difference between AWT and Swing?

AWT (Abstract Window Toolkit) uses native operating-system components and is heavyweight.

Swing is built on top of AWT and provides lightweight, richer and more flexible GUI components.

Examples:

AWT – Frame, Button, Label.

Swing – JFrame, JButton, JLabel.

10. List any three commonly used Swing components with purpose.

JFrame – top-level application window.

JLabel – displays text or an image.

TextField – accepts single-line text input.

JButton – creates a clickable button.

JMenuBar – holds menus at the top of a JFrame.

PART C – Long Answers / Programming

1. Console Input Using Scanner

Java provides several ways to read console input, including Scanner and BufferedReader. Scanner is simple and supports methods for different data types.

Program:

```
import java.util.Scanner;

class ConsoleInput {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter integer: ");

        int n = sc.nextInt();

        System.out.print("Enter double: ");

        double d = sc.nextDouble();

        sc.nextLine(); // consume newline

        System.out.print("Enter string: ");

        String s = sc.nextLine();

        System.out.println("Integer = " + n);

        System.out.println("Double = " + d);
```

```

        System.out.println("String = " + s);

        sc.close();

    }

}

```

The Scanner object reads values from System.in using nextInt(), nextDouble() and nextLine().

2. BufferedReader and BufferedWriter – File Handling

BufferedReader reads text efficiently, while BufferedWriter writes text efficiently using buffering.

Program:

```

import java.io.*;

class ReverseFile {

    public static void main(String[] args) throws Exception {

        BufferedReader br = new BufferedReader(new FileReader("input.txt"));

        BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"));

        java.util.ArrayList<String> lines = new java.util.ArrayList<>();

        String line;

        while ((line = br.readLine()) != null) {

            lines.add(line);

        }

        for (int i = lines.size() - 1; i >= 0; i--) {

            bw.write(lines.get(i));

            bw.newLine();

        }

        br.close();

        bw.close();
    }
}

```

```

        System.out.println("File copied in reverse order");
    }
}

```

The program reads all lines into an ArrayList and writes them from last to first into another file.

3. File Class – Methods and Program

The File class in java.io represents a file or directory pathname.

Common methods:

exists() – checks existence.

getName() – returns file name.

length() – returns size in bytes.

lastModified() – returns modification time.

isFile() – checks whether it is a file.

isDirectory() – checks whether it is a directory.

Program:

```

import java.io.*;
import java.util.Date;

class FileInfo {

    public static void main(String[] args) {

        File f = new File("input.txt");

        if (f.exists()) {

            System.out.println("Name: " + f.getName());

            System.out.println("Size: " + f.length() + " bytes");

            System.out.println("Last Modified: " + new Date(f.lastModified()));

        } else {

            System.out.println("File does not exist");
        }
    }
}

```

```
    }  
}  
}
```

4. Swing Architecture and Simple GUI

Swing is a GUI toolkit in the javax.swing package. A Swing application commonly uses a top-level container such as JFrame and components such as JLabel, JTextField and JButton.

Program:

```
import javax.swing.*;  
  
class SimpleSwing {  
    public static void main(String[] args) {  
        JFrame frame = new JFrame("My Application");  
        JLabel label = new JLabel("Enter Name:");  
        label.setBounds(30, 30, 100, 30);  
        JTextField text = new JTextField();  
        text.setBounds(130, 30, 150, 30);  
        JButton button = new JButton("Submit");  
        button.setBounds(100, 80, 100, 30);  
  
        frame.add(label);  
        frame.add(text);  
        frame.add(button);  
        frame.setSize(350, 180);  
        frame.setLayout(null);  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        frame.setVisible(true);  
    }  
}
```

```
}  
  
}
```

Components are added to the JFrame, the layout is defined, the size is set and finally the window is made visible.

5. Creating Menus in Java Swing

A Swing menu system commonly contains a JMenuBar, JMenu and JMenuItem. JMenuBar is attached to a JFrame. JMenu contains menu items.

Program:

```
import javax.swing.*;  
  
class MenuDemo {  
  
    public static void main(String[] args) {  
  
        JFrame frame = new JFrame("Menu Example");  
  
        JMenuBar bar = new JMenuBar();  
  
        JMenu file = new JMenu("File");  
  
        JMenuItem open = new JMenuItem("Open");  
  
        JMenuItem save = new JMenuItem("Save");  
  
        JMenuItem exit = new JMenuItem("Exit");  
  
        file.add(open);  
  
        file.add(save);  
  
        file.add(exit);  
  
        bar.add(file);  
  
        frame.setJMenuBar(bar);  
  
        frame.setSize(400, 250);  
  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
  
        frame.setVisible(true);  
  
    }  
}
```

}

Structure:

JFrame

└─ JMenuBar

└─ JMenu (File)

└─ JMenuItem (Open)

└─ JMenuItem (Save)

└─ JMenuItem (Exit)

Exam Tip: For 10/12-mark questions, write definition, key points, syntax/concept, program, output, and conclusion wherever applicable.